

验收指导

这份文档的作用

这份文档是给你在课程验收、自己复盘、或后续继续调试时使用的。

目标是让你可以：

1. 从一个干净目录开始重新编译项目
2. 运行基础测试脚本
3. 分阶段验证每一类功能
4. 快速定位编译问题、语法问题、执行问题、存储问题
5. 理解当前哪些功能已经稳定，哪些属于改进方向

一、准备环境

推荐环境： WSL / Ubuntu

安装依赖：

```
sudo apt update
sudo apt install -y gcc make flex bison
```

进入项目目录：

```
cd /mnt/d/CodeFource/School/testDBMS/dbms
```

二、从头开始的标准验收流程

第 1 步：清理旧产物

```
make clean
make reset
mkdir -p data
touch data/.gitkeep
```

作用：

- 删除上次编译生成的 `dbms`、`lex.yy.c`、`sql.tab.c`、`sql.tab.h`
- 删除运行时数据目录内容
- 准备一个干净的 `data/`

如果你只想最小清理，也可以直接：

```
make clean
rm -rf data
mkdir -p data
touch data/.gitkeep
```

第 2 步：重新编译

```
make
```

预期现象：

- 会自动运行 `bison`
- 会自动运行 `flex`
- 会自动调用 `gcc`
- 最终生成可执行文件 `dbms`

如果编译失败，优先看：

- `sql.y` 是否有语法动作错误
- `sql.l` 是否有词法规则错误
- `executor.c` / `storage.c` 是否有函数声明、静态函数顺序或类型不一致问题

第 3 步：运行基础总测试

```
./dbms test/test.sql
```

这是最重要的一步，因为它覆盖了实验主体的 11 类基础 SQL 功能。

当前版本在干净状态下应该能完整跑通。

三、基础功能分阶段验收

建议你不要只跑总脚本，还要能拆开验证。这样老师问到某个功能时，你可以马上证明。

阶段 1：数据库操作

测试命令：

```
cat > /tmp/t1.sql << 'EOF'
CREATE DATABASE XJGL;
CREATE DATABASE JUST_FOR_TEST;
CREATE DATABASE JUST_FOR_TEST;
SHOW DATABASES;
DROP DATABASE JUST_FOR_TEST;
SHOW DATABASES;
USE XJGL;
EOF
./dbms /tmp/t1.sql
```

重点检查：

- 建库是否成功
- 重复建库是否报错
- SHOW DATABASES 是否正确
- 删库后是否消失
- USE XJGL 是否成功

涉及功能文件：

- [sql.y](#)
- [executor.c](#)
- [dbms.c](#)

阶段 2：表操作

测试命令：

```
cat > /tmp/t2.sql << 'EOF'
USE XJGL;
CREATE TABLE STUDENT(SNAME CHAR(20),SAGE INT,SSEX INT);
CREATE TABLE COURSE(CNAME CHAR(20),CID INT);
SHOW TABLES;
DROP TABLE COURSE;
SHOW TABLES;
EOF
./dbms /tmp/t2.sql
```

重点检查：

- 建表是否成功
- SHOW TABLES 是否列出正确表名
- DROP TABLE 后是否从系统表中移除

涉及功能文件：

- [sql.y](#)
- [executor.c](#)
- [storage.c](#)

阶段 3：INSERT + 单表 SELECT

测试命令：

```
cat > /tmp/t3.sql << 'EOF'
USE XJGL;
CREATE TABLE STUDENT(SNAME CHAR(20),SAGE INT,SSEX INT);
INSERT INTO STUDENT VALUES ('ZHANGSAN',22,1);
INSERT INTO STUDENT VALUES ('LISI',23,0);
INSERT INTO STUDENT(SNAME,SAGE) VALUES ('WANGWU',21);
SELECT * FROM STUDENT;
SELECT SNAME,SAGE FROM STUDENT WHERE SAGE=21;
SELECT SNAME FROM STUDENT WHERE (SAGE>21) AND (SSEX=0);
EOF
./dbms /tmp/t3.sql
```

重点检查：

- 全列插入是否正确
- 缺列插入是否补默认值
- `SELECT *` 是否显示全部记录
- 条件查询是否正确

涉及功能文件：

- [executor.c](#)
- [storage.c](#)

阶段 4：多表 SELECT

测试命令：

```
cat > /tmp/t4.sql << 'EOF'
USE XJGL;
SELECT * FROM STUDENT;
SELECT * FROM COURSE;
SELECT * FROM STUDENT,COURSE;
SELECT * FROM STUDENT,COURSE WHERE (SSEX=0) AND (CID=1);
EOF
./dbms /tmp/t4.sql
```

重点检查：

- 是否能做多表笛卡尔积
- `WHERE` 条件是否能正确过滤多表结果

涉及功能文件：

- [executor.c](#)

阶段 5：DELETE + UPDATE

测试命令：

```
cat > /tmp/t5.sql << 'EOF'
USE XJGL;
SELECT * FROM STUDENT;
DELETE FROM STUDENT WHERE (SAGE>21) AND (SSEX=0);
SELECT * FROM STUDENT;
UPDATE STUDENT SET SAGE=21 WHERE SSEX=1;
SELECT * FROM STUDENT;
UPDATE STUDENT SET SAGE=27,SSEX=1 WHERE SNAME='ZHANGSAN';
SELECT * FROM STUDENT;
EOF
./dbms /tmp/t5.sql
```

重点检查：

- 删除前后记录是否正确
- 更新行数是否合理
- 多列更新是否生效

涉及功能文件：

- [executor.c](#)
- [storage.c](#)

四、健壮性与边界情况验收

1. 错误语法恢复

测试命令：

```
echo "CREATE TABLE BAD(COL1 CHAR(22));" | ./dbms
```

当前预期：

```
Syntax error near ';' (skipped).
```

说明：

- 这是通过 `sql.y` 中的 `error ';' 恢复实现的`
- 重点不是报错文字多漂亮，而是后续语句不能被卡死

2. 小写关键字

测试命令：

```
cat > /tmp/lower.sql << 'EOF'
CREATE DATABASE D1;
USE D1;
CREATE TABLE T(ID INT);
insert into t values (1);
select * from t;
EOF
./dbms /tmp/lower.sql
```

重点检查：

- 小写 insert
- 小写 select
- 小写表名 / 大小写统一处理

3. 同一行多条语句

测试命令：

```
cat > /tmp/multi.sql << 'EOF'
CREATE DATABASE D2;USE D2;CREATE TABLE T(ID INT);SHOW TABLES;
EOF
./dbms /tmp/multi.sql
```

重点检查：

- 是否能顺序执行
- 不应因为同一行有多个语句而只执行第一个

4. 缺列插入

测试命令：

```
cat > /tmp/default.sql << 'EOF'
CREATE DATABASE D3;
USE D3;
CREATE TABLE T(A CHAR(10),B INT,C INT);
INSERT INTO T(A,B) VALUES ('X',1);
SELECT * FROM T;
EOF
./dbms /tmp/default.sql
```

重点检查：

- c 是否被补成 0

5. 多层括号 WHERE

测试命令：

```
cat > /tmp/where.sql << 'EOF'
CREATE DATABASE D4;
USE D4;
CREATE TABLE T(ID INT);
INSERT INTO T VALUES (1);
SELECT * FROM T WHERE (((ID=1)));
EOF
./dbms /tmp/where.sql
```

重点检查：

- 多层括号不会造成语法错误
- 条件结果正确

五、改进方向如何验收

1. 安全

测试命令：


```
cat > /tmp/auth.sql << 'EOF'
CREATE DATABASE SEC;
USE SEC;
CREATE TABLE T(ID INT);
INSERT INTO T VALUES (1);
CREATE USER ALICE;
GRANT SELECT ON T TO ALICE;
LOGIN ALICE;
SELECT * FROM T;
INSERT INTO T VALUES (2);
EOF
./dbms /tmp/auth.sql
```

预期：

- ALICE 能 SELECT
- ALICE 不能 INSERT

调试重点：

- [executor.c](#) 中的权限检查路径

2. 视图

测试命令：

```
cat > /tmp/view.sql << 'EOF'
CREATE DATABASE V;
USE V;
CREATE TABLE P(ID INT,NAME CHAR(10));
INSERT INTO P VALUES (1,'A');
CREATE VIEW VP AS SELECT ID,NAME FROM P WHERE ID=1;
SHOW VIEWS;
SELECT * FROM VP;
EOF
./dbms /tmp/view.sql
```

当前可验收点：

- CREATE VIEW
- SHOW VIEWS
- SELECT * FROM VP

注意：

- 当前更推荐验收 `SELECT * FROM 视图`
- 不建议把“视图上的复杂再次查询”作为当前版本的强验收点

3. 索引 / 主码 / 外码 / 约束

测试命令：

```
cat > /tmp/constraint.sql << 'EOF'
CREATE DATABASE C1;
USE C1;
CREATE TABLE P(ID INT PRIMARY KEY, NAME CHAR(10) UNIQUE);
CREATE TABLE C(ID INT, PID INT, FOREIGN KEY (PID) REFERENCES P(ID));
INSERT INTO P VALUES (1,'A');
INSERT INTO P VALUES (1,'B');
INSERT INTO C VALUES (1,2);
INSERT INTO C VALUES (2,1);
CREATE INDEX IDX_P_ID ON P(ID);
SELECT * FROM P WHERE ID=1;
EOF
./dbms /tmp/constraint.sql
```

预期：

- 第二条插入 `P` 会被主码冲突拒绝
- 第一条插入 `C` 会被外码约束拒绝
- 第二条插入 `C` 成功
- 建索引成功

4. 事务 / 日志

测试命令：

```
cat > /tmp/txn.sql << 'EOF'
CREATE DATABASE TX;
USE TX;
CREATE TABLE T(ID INT,NAME CHAR(10));
INSERT INTO T VALUES (1,'A');
BEGIN;
UPDATE T SET NAME='B' WHERE ID=1;
SELECT * FROM T;
ROLLBACK;
SELECT * FROM T;
BEGIN;
UPDATE T SET NAME='C' WHERE ID=1;
COMMIT;
SELECT * FROM T;
EOF
./dbms /tmp/txn.sql
```

预期：

- 第一次事务里临时查询应看到 B
- 回滚后应恢复为 A
- 第二次事务提交后应保留为 C

六、出现问题时该从哪里调

编译不过

先看：

- [sql.y](#)
- [sql.l](#)
- [executor.c](#)

常见原因：

- 新增函数没声明
- 静态函数顺序和声明冲突
- Bison 语义值类型不匹配
- Flex 规则写坏

能编译但语法报错

先看：

- `sql.l`
- `sql.y`

如果是关键字识别异常：

- 查 `to_upper()` 处理
- 查关键字映射是否补全

如果是括号、逗号、； 报错：

- 查 `sql.y` 的规则定义

语法通过但执行结果不对

先看：

- `executor.c`

定位思路：

- 数据库/表级问题：看建库建表、系统表写入
- 插入查询问题：看 `exec_insert` / `exec_select`
- 删除更新问题：看 `exec_delete` / `exec_update`
- 授权问题：看 `check_table_priv`
- 事务问题：看 `append_txn_log_record`、事务日志应用

文件落盘不对

先看：

- `storage.c`
- `data/` 目录内容

建议直接查看：

```
cat data/sys.dat
cat data/XJGL/sys.dat
cat data/XJGL/STUDENT.dat
```

七、推荐的最终验收命令顺序

如果老师现场让你从零演示，我建议你按下面顺序走：

```
cd /mnt/d/CodeFource/School/testDBMS/dbms
make clean
make reset
mkdir -p data
touch data/.gitkeep
make
./dbms test/test.sql
```

如果老师要求你分功能讲解，就按本文件第三部分的 5 个阶段分别跑。

如果老师追问改进方向，就按本文件第五部分分别跑 4 组脚本。

八、你现在最需要记住的几件事

1. 基础 11 类 SQL 功能已经能完整跑通。
2. test/test.sql 是当前最重要的总验收脚本。
3. 错误恢复、小写关键字、缺列插入、多层括号条件都已经处理过。
4. 改进方向里：
 - 安全：基本可演示
 - 视图：基础可演示
 - 约束与索引：基本可演示
 - 事务：基础提交/回滚可演示
5. 如果现场要快速证明系统是好的，最有说服力的命令就是：

```
make clean && make
./dbms test/test.sql
```